

"Something Big Is Happening" at AMN

Embracing the AI Engineering Paradigm at AMN Healthcare

Presenter: Michael Lindemuth, Principal AI Solutions Architect **AI:** Opus 4.5 & 4.6

Part 1: "Something Big Is Happening"

~15 minutes

Slide 1: The Shift

INCOMING

March 2026

SPEAKER NOTES

Welcome everyone. I want to start with a simple statement: something big is happening in software engineering right now. Not "coming soon." Not "on the horizon." It's happening **right now**, and it's changing how software gets built in fundamental ways. Over the next 45 minutes, I want to walk you through what's changed, why it matters to AMN Healthcare specifically, what our new engineering workflow looks like, and — honestly — some of the hard truths about this transition. Let's dive in.

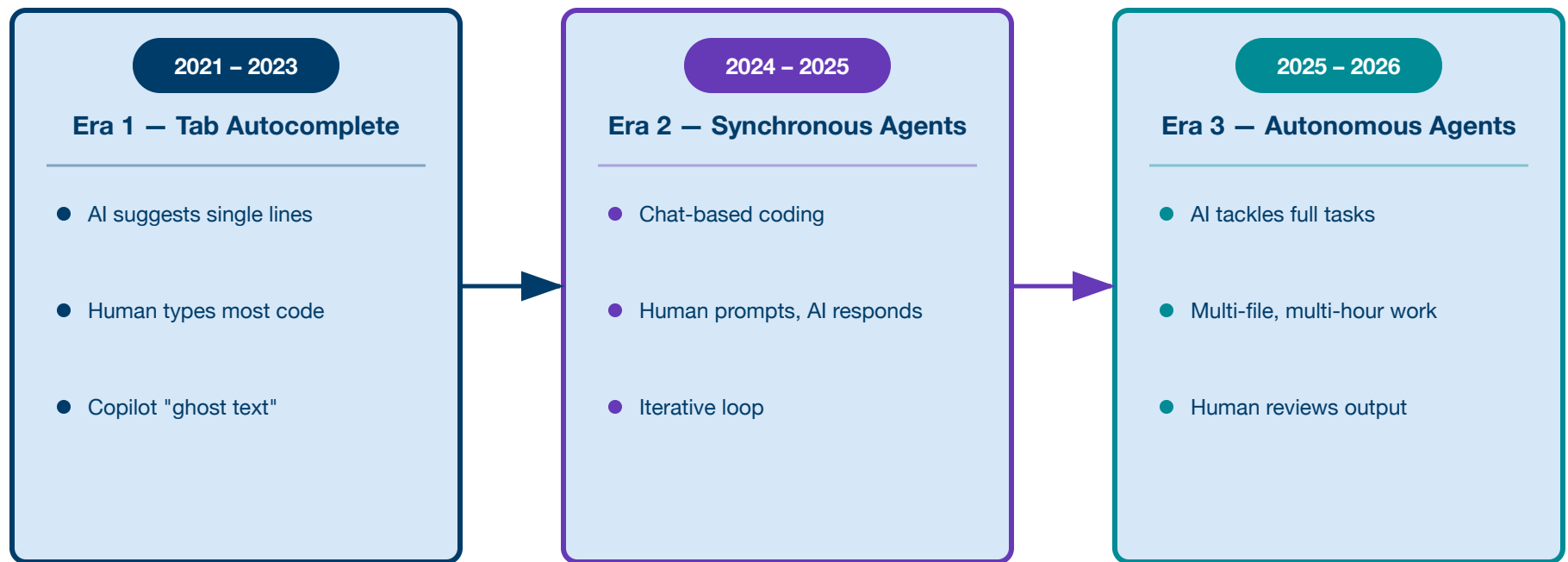
Slide 2: The Three Eras of AI Software Development

Source: Michael Truell, CEO of Cursor — cursor.com/blog/third-era

- **Era 1: Tab Autocomplete** — AI suggests line completions, human types
- **Era 2: Synchronous Agents** — Human prompts, AI responds in a conversational loop
- **Era 3: Autonomous Agents** — AI tackles large, multi-file tasks independently over hours

The Numbers (Cursor, 2025-2026):

Metric	Value
Agent usage growth	15x in one year
Agent vs. tab users	2x more agent users than tab users
Agent-created PRs	1 in 3 Cursor PRs are agent-created



Progression of AI in Software Development

SPEAKER NOTES

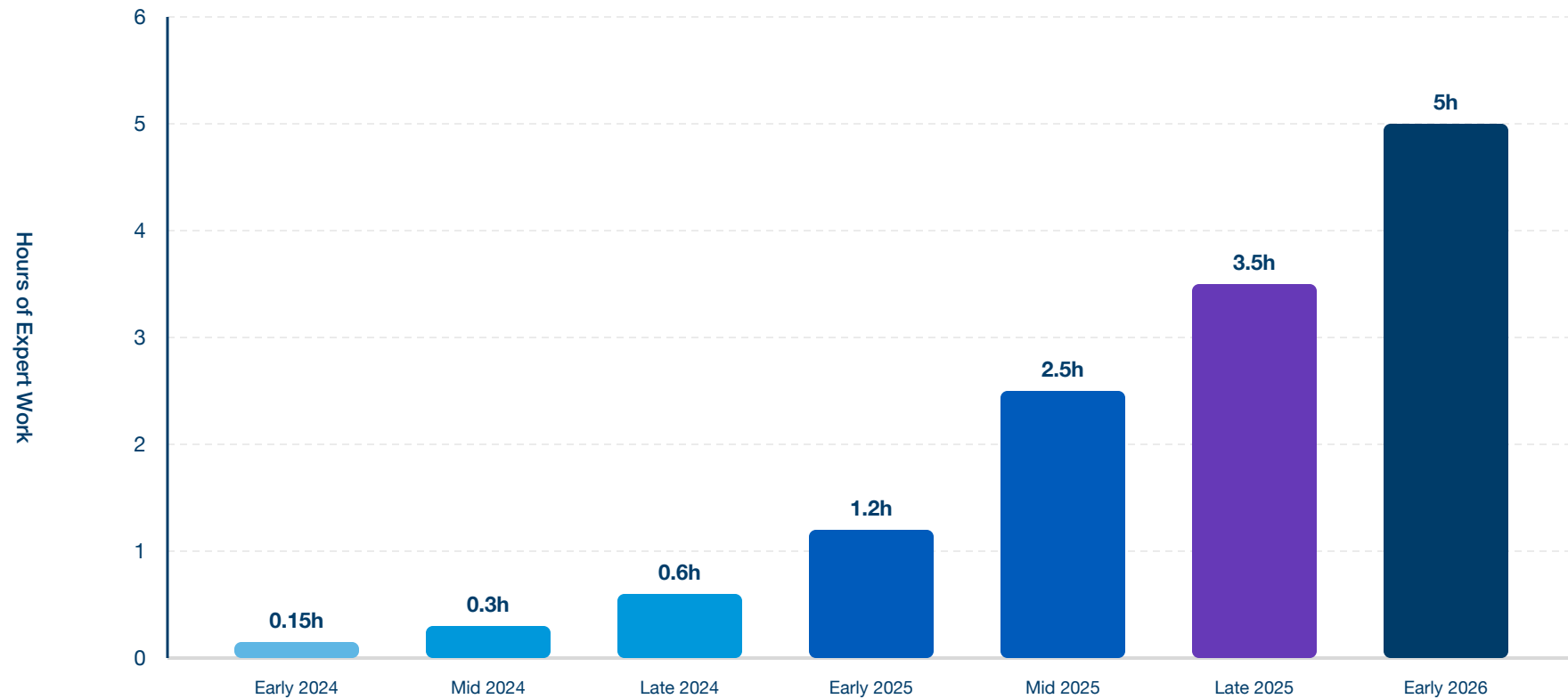
Michael Truell, CEO of Cursor, laid out this framework. In Era 1, AI was basically a fancy autocomplete — you'd see a gray "ghost text" suggestion and hit Tab. Useful, but incremental. In Era 2, we got chat-based agents — you could describe what you wanted, and the AI would write code blocks for you in a conversational loop. That's where most of us have been living. But Era 3 is where things get interesting. Autonomous agents can take a specification, plan their approach, implement across multiple files, run tests, and iterate — all without you typing a single line of code. At Cursor, agent usage grew 15x in a single year. They now have twice as many agent users as tab-completion users. And one in every three pull requests at Cursor is now created by an agent. That's not a toy — that's a fundamental shift in how code gets written.

Slide 3: What Changed

Source: Matt Shumer — "[Something Big Is Happening](#)"

- Models crossed a **capability threshold** in early 2026
 - GPT-5.3 Codex, Claude Opus 4.6
- AI now demonstrates "**judgment**" and "**taste**" — not just instruction-following
- Developers describe outcomes in **plain English** → AI builds complete applications
- METR tracking: autonomous task duration grew from **~10 minutes** to **~5 hours** of expert-equivalent work

AI Autonomous Task Duration (Expert-Equivalent Hours)



Source: METR AI capability tracking

Source: [METR AI capability tracking](#), cited in Shumer

SPEAKER NOTES

Matt Shumer wrote an essay that perfectly captures what happened. The models didn't just get incrementally better — they crossed a threshold. GPT-5.3 Codex and Claude Opus 4.6 don't just follow instructions. They demonstrate something that looks a lot like judgment. They make architectural decisions. They anticipate edge cases. They refactor when things get messy. METR, which tracks AI capability benchmarks, shows that the autonomous task duration — meaning how long an AI can work independently on expert-level tasks — grew from about 10 minutes to roughly 5 hours of equivalent expert work. That's not autocomplete. That's a junior-to-mid-level engineer working a half-day on a problem.

Slide 4: The Self-Improvement Loop

- OpenAI: GPT-5.3 "**was instrumental in creating itself**"
- AI is contributing to its own:
 - Training pipeline development
 - Debugging and testing
 - Deployment automation
- Capability doubling every **~7 months**, potentially accelerating to **~4 months**

SPEAKER NOTES

Here's where it gets really interesting — and, frankly, a little surreal. OpenAI stated publicly that GPT-5.3 was instrumental in creating itself. The model contributed to its own training infrastructure, debugging, and deployment pipeline. We're entering a self-improvement loop where AI capabilities compound. Current estimates show capability doubling roughly every 7 months, with some researchers projecting that could accelerate to 4 months. What this means practically: the AI tools you're using today will be significantly more capable in 6 months. The workflow we're going to discuss isn't just about today — it's designed to scale with what's coming.

Slide 5: Why This Matters

Source: Citrini Research — "*The 2028 Global Intelligence Crisis*"

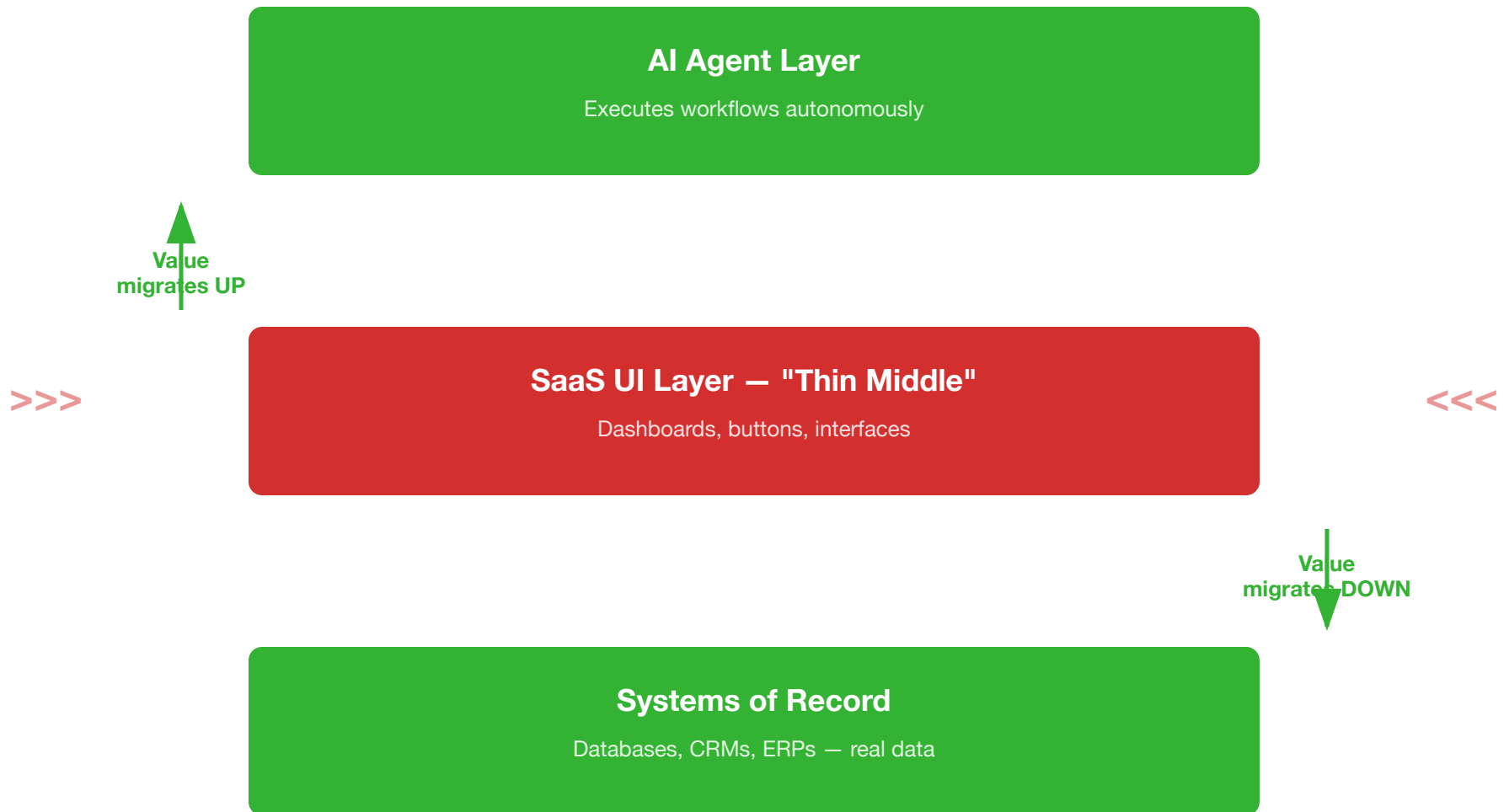
- Competent developers can now **replicate mid-market SaaS products in weeks**
- The competitive landscape is shifting:
 - **Speed** becomes the primary differentiator
 - **Specification quality** determines output quality
- Economic analysts project significant disruption to traditional software business models
- **The engineers who adapt will thrive; those who don't face real risk**

SPEAKER NOTES

So why does this matter to us specifically? Because the competitive landscape is shifting underneath everyone. Citrini Research published a paper on what they call the "2028 Global Intelligence Crisis" — their thesis is that when capable developers can replicate mid-market SaaS products in weeks instead of months, the entire economics of the software industry change. Speed becomes the differentiator. And what determines speed? The quality of your specifications and the effectiveness of your AI-augmented workflow. I'm not saying this to alarm anyone. I'm saying it because the engineers who lean into this transition will be extraordinarily productive. And those who treat AI tools as optional are going to find themselves at a significant disadvantage.

Slide 6: The Thin Middle Squeeze

Source: David Ondrej — x.com/DavidOndrej1/status/2019126831761572169



Real-World Impact:

Company	Forward P/E (Peak)	Forward P/E (Current)
Adobe	30	12
ServiceNow	67	28

SPEAKER NOTES

David Ondrej articulated something I think is critical for understanding where value is moving. Picture the software stack in three layers. At the top, you have the AI agent layer — systems that autonomously execute workflows. At the bottom, you have systems of record — databases, CRMs, ERPs — where the actual data lives. In the middle, you have the SaaS UI layer — the dashboards, the buttons, the pretty interfaces that sit on top of someone else's data. Here's the squeeze: value is migrating UP into the agent layer and DOWN into the data layer. The middle — software whose entire value proposition was "we put a nice UI over your data" — gets crushed. Look at the market: Adobe's forward P/E dropped from 30 to 12. ServiceNow went from 67 to 28. Investors are repricing these companies because agents don't need pretty dashboards. They need APIs, clean data, and workflow automation.

Slide 7: Where Value Moves

Value is concentrating in:

1. **AI platform subscriptions** — usage-based, not per-seat pricing
2. **Systems of record** — agents need clean, authoritative data
3. **Security, governance, and compliance infrastructure** — the guardrails
4. **Outcome-based pricing** — "\$5 per contract reviewed" vs "\$99/seat/month"
5. **Services** — implementation, customization, integration (complexity lives here)

"SaaS as a delivery model is fine. SaaS as a business strategy built on shallow moats and per-seat pricing — that's over."

SPEAKER NOTES

So where does value actually move? Five places. First, AI platform subscriptions — but priced on usage, not per-seat. Second, systems of record — agents need clean, authoritative data to operate. Garbage in, garbage out applies even more with AI. Third, security and governance infrastructure — somebody has to build the guardrails. Fourth, outcome-based pricing — instead of "\$99 per seat per month," you'll see "\$5 per contract reviewed" or "\$0.10 per invoice processed." And fifth, services — implementation, customization, integration work. The complexity doesn't go away; it just changes form. The quote I want you to remember: "SaaS as a delivery model is fine. SaaS as a business strategy built on shallow moats and per-seat pricing — that's over."

Slide 8: What This Means for AMN Engineers

- Frontier models are making **"build" cheaper than "buy"** — especially where vendors occupy the **"thin middle"** with little defensible value
- We build **systems of record** and **agent-layer tooling** — we own the layers that actually matter
- Our value is in the **data**, the **domain knowledge**, and the **execution layer** — not in commodity UI wrappers
- Engineers who can **specify**, **architect**, and **review** at the agent + data layers will be in highest demand
- The workflow we're about to discuss positions you there

SPEAKER NOTES

So what does all of this mean for us at AMN? Here's the big shift: frontier AI models are flipping the traditional build vs. buy equation on its head. It used to be that buying packaged software was almost always cheaper and faster than building. But now, when a vendor's product is mostly a thin layer of UI sitting between you and a capability that AI can deliver directly, building your own becomes the smarter, cheaper option. These "thin middle" vendors — the ones whose entire value prop is wrapping an API or orchestrating a workflow that an LLM can now handle — are the most vulnerable. And we're not one of them. We build and maintain systems of record — the actual data infrastructure that powers healthcare staffing. We build the agent-layer tooling and the integrations. Our value is in our data, our domain knowledge, and our execution layer. The engineers who will be in highest demand — both at AMN and industry-wide — are those who can specify systems clearly, architect solutions at the agent and data layers, and review AI-generated output with discernment. That's exactly what the workflow I'm about to walk you through is designed to develop.

Part 2: "The New AMN Engineering Workflow"

~20 minutes

Slide 9: From Writing Code to Guiding Agents

- You still **own the outcome** — correctness, architecture, quality
- But instead of typing every line, you **direct AI agents** to implement your vision
- Your engineering judgment becomes the **input**, not the keystrokes
- The workflow that follows is how we do this **systematically** at AMN

SPEAKER NOTES

So we've talked about what's happening in the industry and why it matters to AMN. Now let's talk about what this actually looks like day-to-day. The fundamental shift is this: your job as an engineer hasn't changed. You're still responsible for building correct, well-architected, high-quality software. What's changed is the method. Instead of sitting down and typing every line of code yourself, you're increasingly directing AI agents to implement your vision. Think of it like the difference between a contractor who swings every hammer and an architect who designs the building and ensures it's built to spec. You still need deep technical knowledge — arguably more than before — but your primary output shifts from code to direction. Your engineering judgment, your understanding of the system, the requirements, the edge cases — that becomes the input. The keystrokes become secondary. Over the next several slides, I'm going to walk you through the specific workflow we've developed at AMN for working this way. It's not abstract — it's a concrete, repeatable process you can start using immediately.

Slide 10: Step 1 — Specification-First Development

- Engineers should spend **MORE** time on specifications, not less
- Specs become the **primary artifact** — they ARE the engineering work
- Reference: **VSDD** (Verified Spec-Driven Development) as an example methodology
 - *Source:* gist.github.com/dollspace-gay/d8d3bc3ecf4188df049d7a4726bb2a00
- Use GitHub Copilot CLI or GitHub Copilot with top-tier models to **iterate on specs**

SPEAKER NOTES

Step 1 is specification-first development. This is counterintuitive for a lot of engineers. You'd think that if AI is writing the code, you'd spend *less* time on specs. The opposite is true. The specification IS your engineering work now. A vague spec produces vague code. A precise spec produces precise code. There's an emerging methodology called Verified Spec-Driven Development — VSDD — that formalizes this. The idea is that you write detailed specifications, verify them against requirements and edge cases, and then use those specs as the input to AI implementation. You can even use AI to help you write better specs. Use Copilot to iterate on your specifications — ask it to poke holes, identify missing edge cases, challenge your assumptions.

Slide 11: What Makes a Good Specification

A good specification includes:

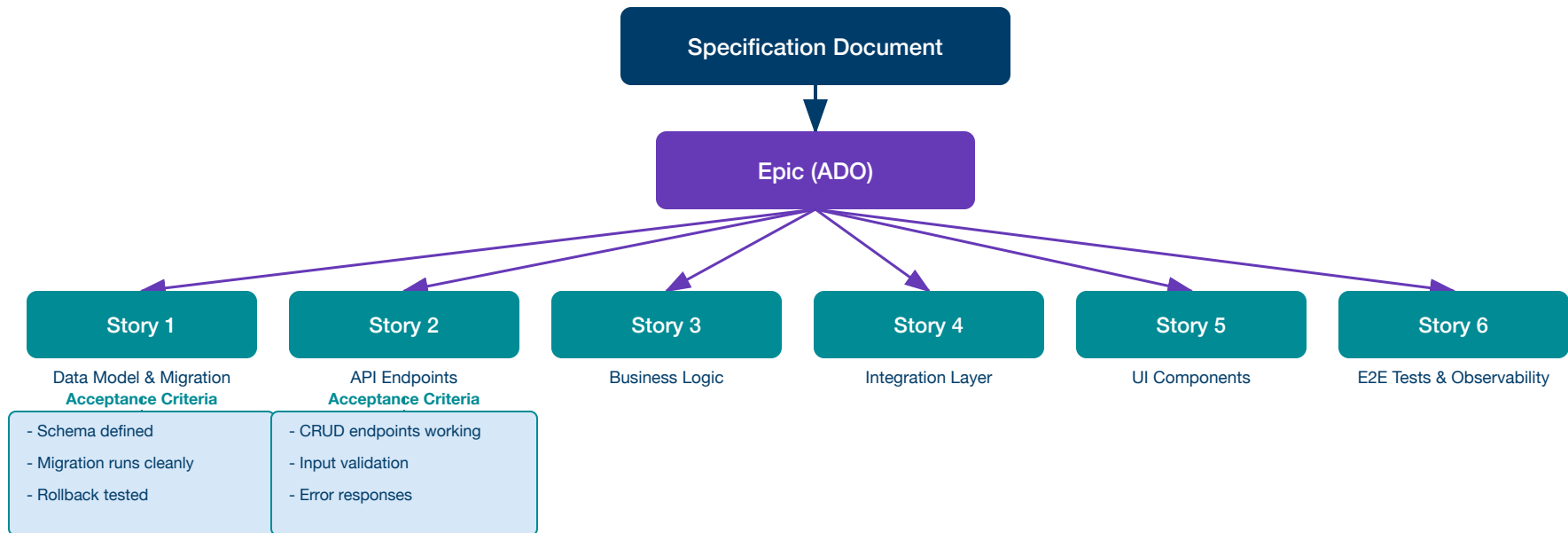
Element	Description
Problem Statement	Clear articulation of what problem we're solving and why
Intended Outcome	What "done" looks like — the end state
Acceptance Criteria	Specific, testable conditions that must be met
Edge Cases	What happens at the boundaries?
API Contracts	Request/response shapes, error codes, versioning
Data Models	Entity relationships, field types, constraints
Integration Points	What other systems does this touch?
Non-Functional Requirements	Performance, security, observability
Existing Patterns	Reference to similar code/patterns in our codebase
Constraints & Out-of-Scope	What we're explicitly NOT doing

SPEAKER NOTES

So what makes a good specification? Here's the checklist I use. You need a clear problem statement — not just "build X" but "solve Y because Z." You need an intended outcome — what does "done" look like? Acceptance criteria that are specific and testable. Edge cases — what happens at the boundaries? API contracts, data models, integration points. Non-functional requirements — how fast does it need to be? What are the security constraints? What observability do we need? References to existing patterns in our codebase — this is critical because it gives the AI context about how we build things here at AMN. And finally, explicit constraints — what are we NOT doing? The better your spec, the better your AI-generated output. Full stop.

Slide 12: Step 2 — Epic & Story Creation with AI

- Use GitHub Copilot or Copilot CLI with the **Azure DevOps MCP server**
- Create an **Epic** linked to the specification
- Break down into multiple **Stories** — each reportable, testable, reviewable
- Stories = units of work humans can reason about and review



SPEAKER NOTES

Step 2: use AI to help create your Epic and Story structure. This is where the Azure DevOps MCP server comes in. You can point Copilot at your specification and say "break this into an Epic with Stories." Each Story should be a reportable, testable, reviewable unit of work. This is important because even though AI is doing the implementation, humans still need to reason about the work. Stories give us that structure. The AI creates the Epic in ADO, links it to your spec, generates Stories with acceptance criteria, and sets up the hierarchy. This isn't just project management — as we'll see in a few slides, this structure becomes critical infrastructure for AI context.

EPIC 488257

488257 Concord Orchestration Layer

Kiley Plett

0 Comments

ClaudeCodeModified X Concord +

Save

Follow

StateActiveAreaAI Platform\AI Digital SolutionsReasonImplementation startedIterationAI Platform\AI Digital Solutions Team

Updated by Kiley Plett: Friday

DetailsScansTime Log Free160

- Blueshift → Concord: Abandonment notifications
- Concord → TAMR: Clinician lookup by phone
- Concord ↔ OneTrust: Consent check and opt-out updates
- Concord ↔ VoiceFlow: Outreach sending, delivery status, opt-out handling, completion notifications
- Concord → AMIE: Call logging for recruiter review

Classification

Value areaBusiness

Mapped on SpecMap

This work item is not mapped on any maps

Add link

[Add an existing work item as a parent](#)

Child

- 489972 Add New Relic Applica... Updated 2h ago Done
- 491286 Concord → VoiceFlow ... Updated Thursday Done
- 488447 Concord | API Key Aut... Updated Feb 12 Done
- 489963 Concord | Restrict Ve... Updated Feb 19 Done
- 491373 Concord | Timed Conv... Updated 3m ago Development

Discussion

Add a comment. Use # to link a work item, @ to mention a person, or ! to link a pull request.

[switch to Markdown editor](#)

Here's what that actually looks like in practice. This is a real Epic in Azure DevOps — "Concord Orchestration Layer," Epic 488257. You can see it's tagged with ClaudeCodeCreated and ClaudeCodeModified, which means AI helped create and refine this work item. Notice the child stories on the right — each one is a discrete,

reviewable unit of work. "Add New Relic Application," "Concord to VoiceFlow," "Concord API Key Authentication" — these are all stories that were broken down from the specification. This is the structure that makes AI-driven development manageable. It's not a black box — every piece of work is tracked, linked, and reviewable.

Slide 14: Story in Azure DevOps

USER STORY 488267

488267 Concord | Integrate with AMIE for Call Logging

JH Jasper Hoong

2 Comments

Save and Close

Follow

ClaudeCodeCreated ClaudeCodeModified Concord

State Done

Area AI Platform\AI Digital Solutions

Reason Moved out of state Deve

Iteration AI Platform\AI Digital Solutions Team\AI Digital Solutions February 2026 Release

Updated by Jasper Hoong: Feb 19

Details Scans Time Log Free Release Notes 16 0

module for AMIE API integration following existing service patterns.

- Implement log call API function: Implement the function to log calls to AMIE via their API.
- Map completion data to AMIE call format: Add AMIE API key and endpoint configuration to the settings/config module.
- Handle AMIE API errors: Define Pydantic schemas for call logging request/response data.

Acceptance Criteria

Click to add Acceptance Criteria.

Discussion

Add a comment. Use # to link a work item, @ to mention a person, or ! to link a pull request.

Service Now ID

Tachyon Work Item Assistant

Improve User Story

Generate Test Cases

Generate Tasks

Planning

Story Points 5

Priority 2

Risk

Development

Add link

GitHub Pull Request

AB#488267 POST API Request to ... #22 merged Feb 19 into 2... Appro...

GitHub Commit

AB#488267 Fix to AMIE integration... Updated Feb 17

AB#488267 Small change to AMIE ... Updated Feb 17

AB#488267 Added Copilot sugges... Updated Feb 17

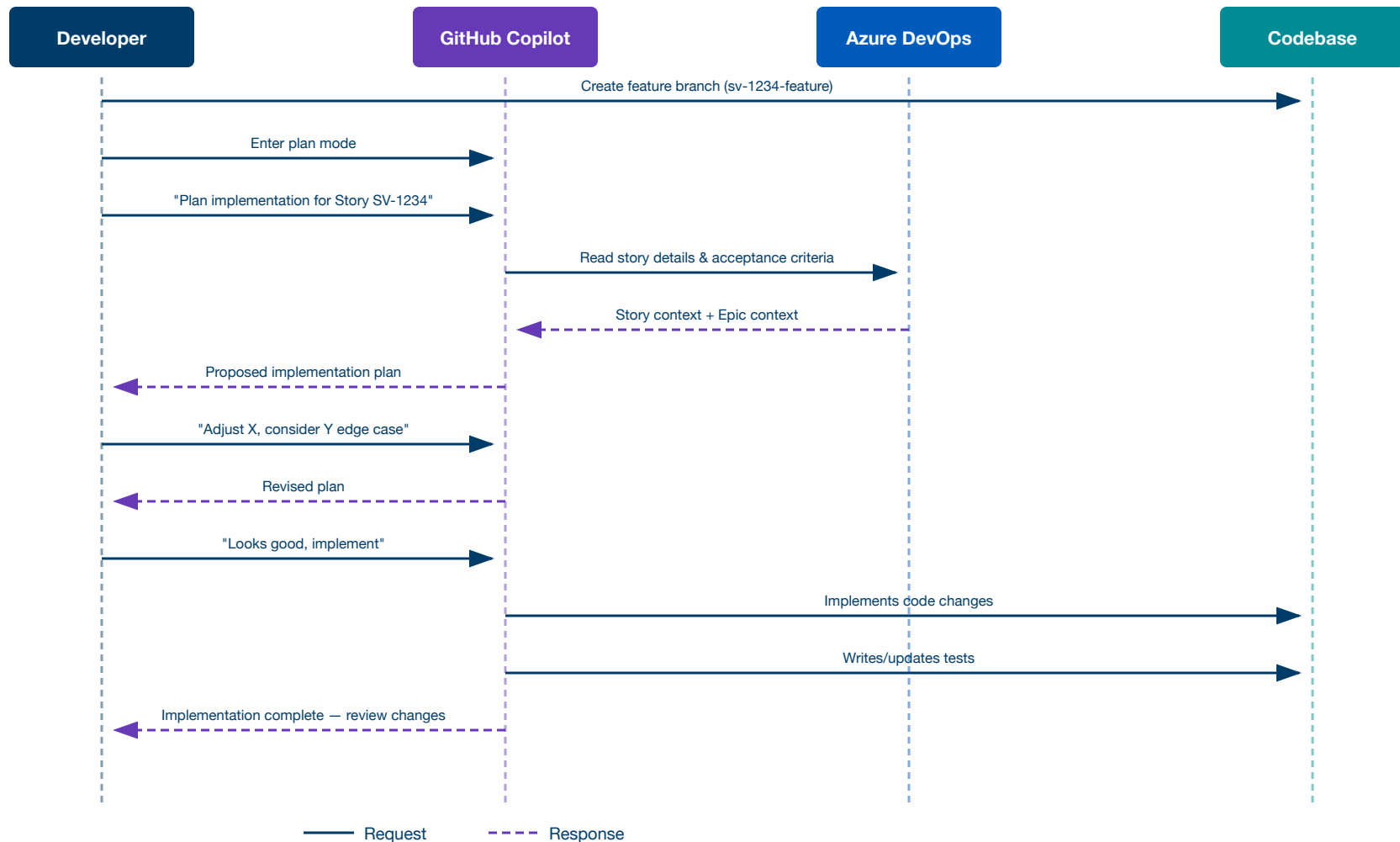
SPEAKER NOTES

And here's what a Story looks like. This is User Story 488267 — "Concord: Integrate with AMIE for Call Logging." Notice it's a child of that Epic we just saw. It's tagged ClaudeCodeCreated and ClaudeCodeModified. Look at the detail here — implementation steps, acceptance criteria, and on the right you can see the linked GitHub pull

requests and commits. This is the level of traceability we're talking about — from specification to Epic to Story to branch to PR to merged code. Every step is visible, reviewable, and auditable.

Slide 15: Step 3 — AI-Driven Implementation

1. Create a **branch** for each story
2. Set GitHub Copilot to **plan mode**
3. Point Copilot at the **specific ADO story**
4. **Refine the plan** collaboratively with Copilot
5. When satisfied, **let Copilot implement the plan**



SPEAKER NOTES

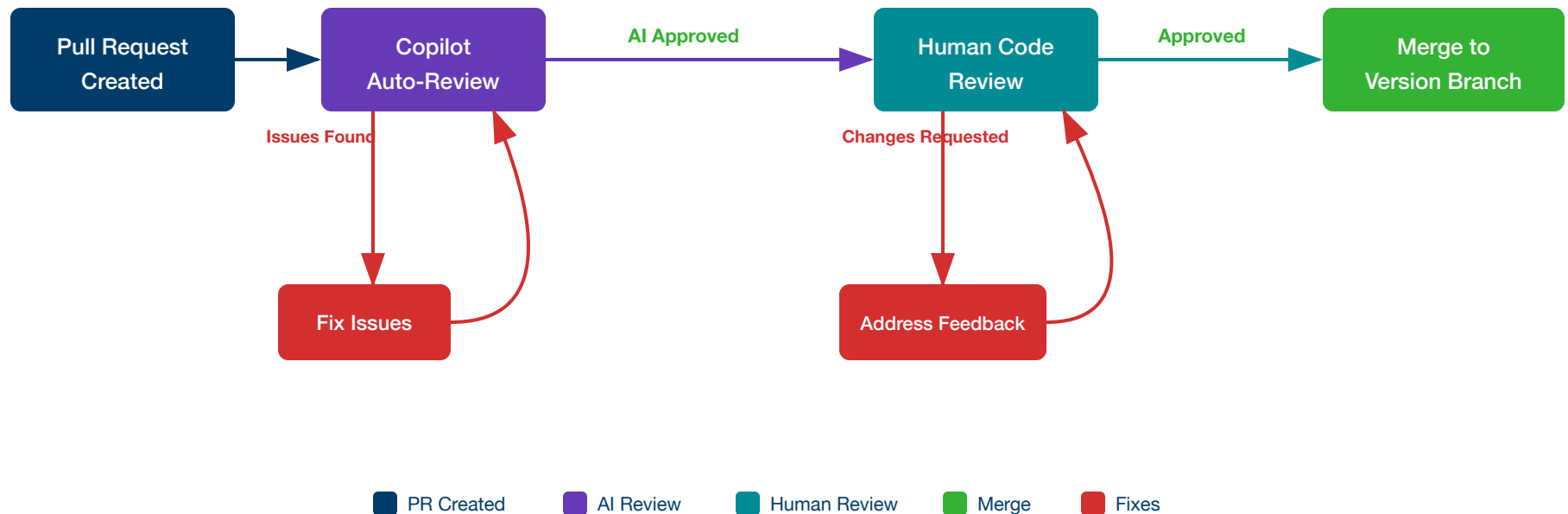
Step 3 is where the magic happens. You create a branch. You open Copilot in plan mode. This is important — plan mode means Copilot doesn't write code yet. It reads the story, reads the codebase, and proposes a plan. You review that plan. You push back. "What about this edge case?" "Use the existing pattern from this module." "Don't forget we need New Relic observability." You iterate on the plan until you're satisfied. THEN you tell Copilot to implement. It writes the code, updates or creates tests, and hands it back to you for review. The key insight: you're not coding. You're **directing**. Your engineering judgment — your knowledge of our systems, our patterns, our requirements — that's what drives the quality of the output.

Slide 16: Step 4 — Review & Merge

1. Configure GitHub Copilot (in GitHub) to **review all PRs automatically**
2. Iterate on Copilot's review feedback until satisfied
3. **Human teammate** reviews both the story and the PR
4. Code only merges after **human approval**

Key Principle: Humans remain responsible for code

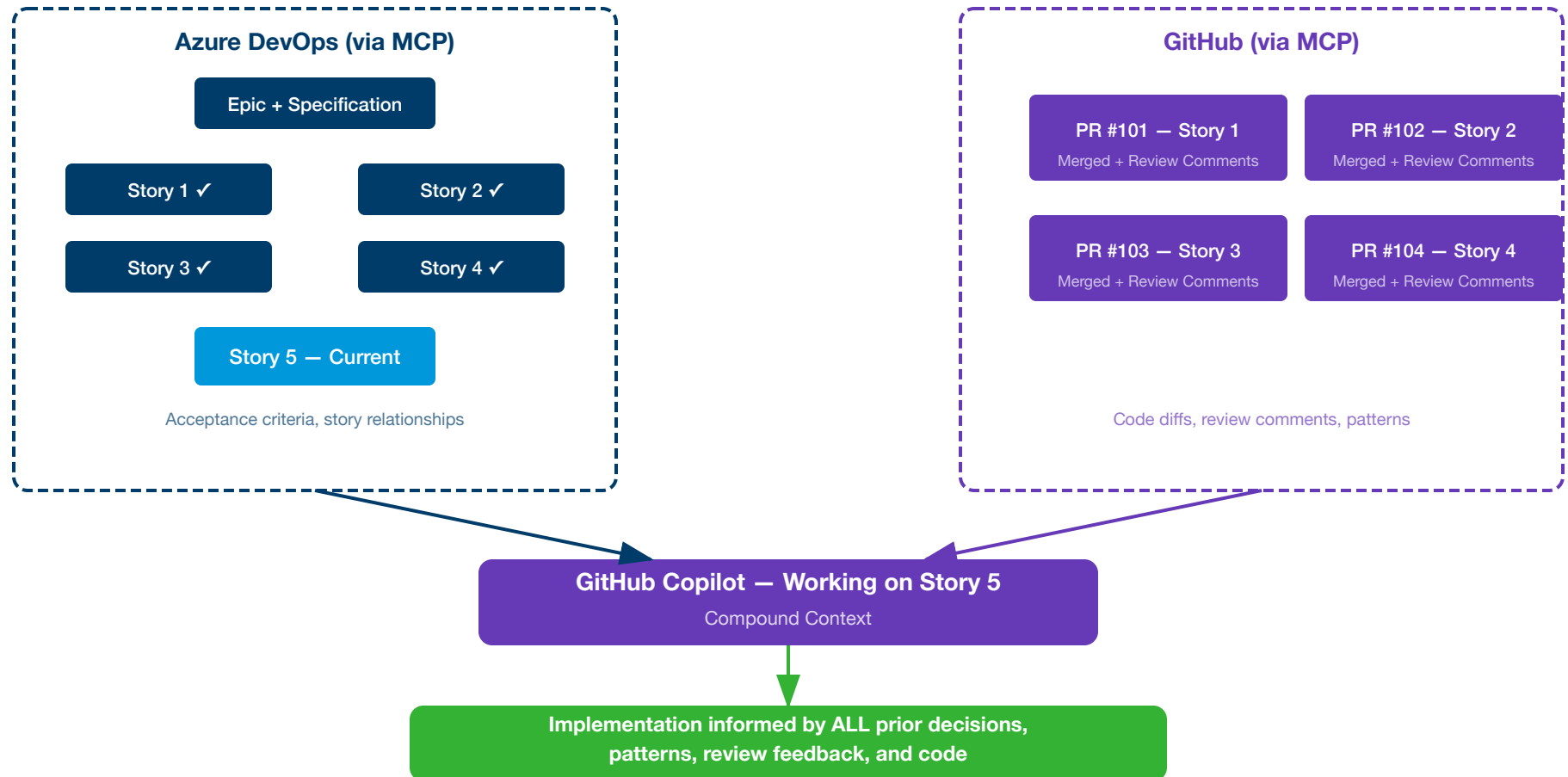
Pull Request Review & Merge Pipeline



SPEAKER NOTES

Step 4: Review and merge. We configure GitHub Copilot to automatically review all pull requests. Copilot catches issues — security vulnerabilities, performance problems, missing test coverage, style inconsistencies. You iterate on that feedback. But here's the critical part: after AI review, a human teammate reviews both the ADO story AND the PR. They verify that the code actually implements what the story requires. Code only merges after human approval. This is non-negotiable. Humans remain responsible for the code that ships. AI is a tool in the process, not the final authority. Your review skills become more important, not less — because you're now reviewing code you didn't write, which requires a different kind of attention.

Slide 17: The Compound Context Advantage

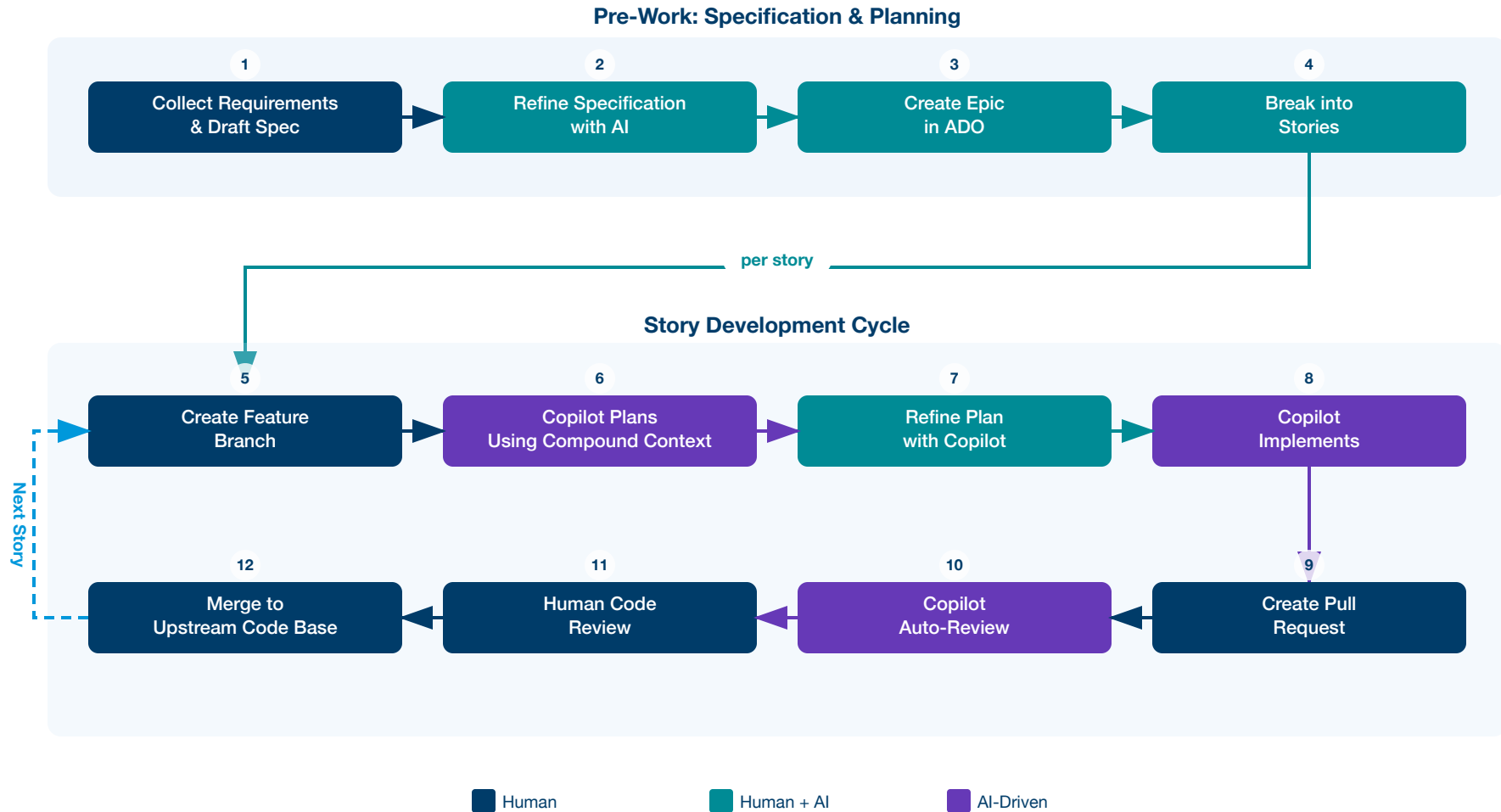


- When PRs link back to ADO stories, and stories link to an Epic, Copilot gains **compound context**
- Via **ADO MCP**: Copilot reads the Epic spec, sibling stories, and acceptance criteria
- Via **GitHub MCP**: Copilot reads merged PRs — code changes, review comments, architectural decisions
- **Result**: Starting Story 5, Copilot already knows what Stories 1-4 built
- This linking structure is **AI context infrastructure**, not just project management

SPEAKER NOTES

This is the slide I'm most excited about because it's where the entire workflow compounds. When your PRs are linked to ADO stories, and those stories are linked to an Epic, Copilot gains what I call "compound context." Through the Azure DevOps MCP server, Copilot can read the entire Epic specification, all sibling stories, and their acceptance criteria. Through the GitHub MCP server, Copilot can read the actual merged PRs from completed stories — the code diffs, the review comments, the architectural decisions that were made. So when you start Story 5, Copilot doesn't start from scratch. It already knows what Stories 1 through 4 built. It knows the patterns that were established. It knows what review feedback was given and how it was addressed. This is why the Epic-to-Story-to-PR linking structure matters so much. It's not just project management — it's AI context infrastructure. The more disciplined you are about this structure, the better your AI output gets on every subsequent story.

Slide 18: The Complete Workflow



Legend:

- Navy: Human activities
- Teal: Human + AI collaboration
- Purple: AI-driven steps

SPEAKER NOTES

Here's the complete workflow end-to-end, split into two clear phases. First is the pre-work — you collect requirements and draft a specification, then you and your AI copilot refine that spec together, create an Epic in ADO, and break the work into Stories. Notice steps 2 through 4 are teal — that means you're collaborating with AI, not doing it alone. Then for each Story you enter the development cycle: create a feature branch, enter Copilot plan mode, refine the plan, let Copilot implement, create a PR, Copilot auto-reviews it, a human reviews it, and it merges. That dashed cyan arrow on the left loops you back to the next Story — but now with all the accumulated context from everything that came before. The color coding is simple: navy is you — the human. Teal is you working together with AI. And purple is AI doing the work. Look at how much purple there is in the cycle versus how much navy. The AI handles planning, implementation, and code review. You handle the git workflow, the human review, and the merge. Your effort concentrates in the pre-work phase and at the review stage — everything in between is increasingly AI-driven.

Slide 19: Copilot PR Review

The screenshot displays a GitHub pull request (PR) titled "AB#488267 POST API Request to AMIE API to log Application Completion #22". The PR is marked as "Merged" and shows it was merged 11 commits into the "AMNEngineering:26.3.1" branch from the "jch1902:AMIE-Integration" branch. A Copilot AI review is visible, indicating it reviewed the PR 2 weeks ago. The review includes a "Pull request overview" section stating that the PR adds AMIE integration to log application completions via VoiceFlow, followed by a list of changes: adding an AMIEClient service class, integrating AMIE logging into the VoiceFlow webhook endpoint, and adding an optional "notes" field to the ApplicationCompleteWebhookRequest schema. Below this, the "Reviewed changes" section notes that Copilot reviewed 7 out of 7 changed files and generated 7 comments. At the bottom, two files are listed: "concord/api/v1/endpoints/voiceflow.py" and "concord/services/amie.py", each with a "Show resolved" link. On the right sidebar, there are options for notifications (Subscribe), participants (3), and a lock conversation button.

AB#488267 POST API Request to AMIE API to log Application Completion #22
jch1902 merged 11 commits into AMNEngineering:26.3.1 from jch1902:AMIE-Integration

Copilot (AI) reviewed 2 weeks ago [View reviewed changes](#)

Copilot (AI) left a comment

Pull request overview

This PR adds AMIE integration to log application completions when clinicians complete their applications via VoiceFlow. The integration follows established patterns in the codebase for external service integration, with a new service client, comprehensive test coverage, and configuration updates.

Changes:

- Added AMIEClient service class with HTTP Basic authentication to POST application completion data to the AMIE API
- Integrated AMIE logging into the VoiceFlow application-complete webhook endpoint with a 60-minute delay for data synchronization
- Added optional `notes` field to ApplicationCompleteWebhookRequest schema to support additional application metadata from VoiceFlow

Reviewed changes

Copilot reviewed 7 out of 7 changed files in this pull request and generated 7 comments.

► Show a summary per file

🔗 Implement all suggestions

👍 👎 🗨

concord/api/v1/endpoints/voiceflow.py [Show resolved](#)

concord/services/amie.py [Show resolved](#)

Notifications [Customize](#)
[Subscribe](#)
You're not receiving notifications from this thread.

3 participants

[Lock conversation](#)

SPEAKER NOTES

Here's a real pull request from the Concord project — PR #22, "POST API Request to AMIE API to log Application Completion." Notice at the top — this PR is already merged. What I want you to focus on is the Copilot AI review. Copilot automatically reviewed this PR, looked at all 7 changed files, and generated 7 comments. It gives you

a pull request overview summarizing the changes, lists the reviewed changes per file, and even offers an "Implement all suggestions" button. This is your first line of defense — AI reviewing AI-generated code. It catches issues before a human ever looks at it.

Slide 20: Human Approval & Merge

The screenshot displays a GitHub pull request interface. At the top, a purple banner indicates the pull request is 'Merged'. The title is 'AB#488267 POST API Request to AMIE API to log Application Completion #22'. Below the title, it shows 'jch1902 merged 11 commits into AMNEngineering:26.3.1 from jch1902:AMIE-Integration'. The commit history shows a commit 'Update tests/services/test_amie.py' with a 'Verified' status. The review process is detailed in the comments: 'Copilot (AI) review requested due to automatic review settings 2 weeks ago', 'Copilot started reviewing on behalf of jch1902 2 weeks ago', and 'Copilot (AI) reviewed 2 weeks ago'. A 'Pull request overview' section states 'Copilot reviewed 7 out of 7 changed files in this pull request and generated no new comments.' and 'Comments suppressed due to low confidence (1)'. A human approval is shown by 'ddb048 approved these changes 2 weeks ago'. The pull request was merged by 'jch1902' with commit '4dfcc44' into 'AMNEngineering:26.3.1' 2 weeks ago, with '6 checks passed'. A final message at the bottom states 'Pull request successfully merged and closed' and provides instructions on deleting the branch.

Merged AB#488267 POST API Request to AMIE API to log Application Completion #22
jch1902 merged 11 commits into `AMNEngineering:26.3.1` from `jch1902:AMIE-Integration`

Update `tests/services/test_amie.py` **Verified** abcf59a

Copilot (AI) review requested due to automatic review settings 2 weeks ago

Copilot started reviewing on behalf of **jch1902** 2 weeks ago [View session](#)

Copilot (AI) reviewed 2 weeks ago [View reviewed changes](#)

Copilot (AI) left a comment

Pull request overview

Copilot reviewed 7 out of 7 changed files in this pull request and generated no new comments.

► Comments suppressed due to low confidence (1)

ddb048 approved these changes 2 weeks ago [View reviewed changes](#)

jch1902 merged commit `4dfcc44` into `AMNEngineering:26.3.1` 2 weeks ago [View details](#) [Revert](#)
6 checks passed

Pull request successfully merged and closed [Delete branch](#)

You're all set — the `AMIE-Integration` branch can be safely deleted. If you wish, you can also delete this fork of `AMNEngineering/concord-backend` in the [settings](#).

SPEAKER NOTES

And here's the rest of the story. After Copilot's initial review, it did a second pass and found no new comments. Then — and this is the critical part — a human teammate, ddb048, approved the changes. Only after that human approval was the PR merged. 8 checks passed, the branch was cleaned up, and the code is in production. This is the workflow in action: AI implements, AI reviews, human approves. The human remains the final authority on what ships.

Slide 21: Generating Reports from ADO Epics

Copilot + ADO MCP = instant status reports for any audience

- Point Copilot at an Epic and ask for a status report:

"Generate a Markdown report on the status of Epic #488257 in ADO. Read related stories and their status in order to generate the report."

- Copilot uses the **ADO MCP server** to fetch the Epic, its child stories, and all statuses
- Specify the **audience** to control the output:
 - *"...for business stakeholders"* → executive summary, progress bars, business value
 - *"...for engineering leadership"* → technical detail, blockers, velocity metrics
- Result: a professional report generated in **seconds**, not hours

Epic 488257: Concord Orchestration Layer

Status Report | February 26, 2026

Executive Summary

The **Concord Orchestration Layer** epic is nearing completion with **87.5% of user stories done** and **~90% of story points completed**. This critical infrastructure enables automated recovery of abandoned clinician applications through intelligent orchestration of marketing events, clinician identity resolution, consent management, and AI-powered conversational outreach.

Metric	Value
Overall Progress	87.5% Complete
Story Points Delivered	46 of 51 (90%)
Priority	Critical (P1)
Owner	Kiley Plett
Created	February 4, 2026

Progress: [=====] 87.5%
14/16 User Stories Complete

Epic Purpose & Business Value

The Concord Orchestration Layer is the backend engine powering AMN Healthcare's **abandoned application recovery program**. When potential clinicians start but don't complete job applications, Concord orchestrates a multi-system workflow to:

1. **Capture abandonment events** from marketing systems (Blueshift)
2. **Resolve clinician identity** through TAMR (identity resolution platform)
3. **Verify communication consent** via OneTrust before any outreach

SPEAKER NOTES

Before we leave Part 2, one more thing that falls naturally out of this workflow. Remember that Epic structure — the one with linked stories and acceptance criteria? Copilot can read all of that context and generate status reports on demand. The prompt is simple: "Generate a Markdown report on the status of Epic 488257 in ADO. Read related stories and their status in order to generate the report." Copilot uses the ADO MCP server to pull the Epic, iterate through every child story, check their statuses and story points, and produce a polished report — complete with an executive summary, metrics table, progress visualization, and business value description. This report shows the Concord Orchestration Layer at 87.5% completion with 90% of story points delivered. Here's the key tip: specify your audience. Say "for business stakeholders" and you get high-level progress and business value. Say "for engineering leadership" and you get technical blockers, velocity metrics, and implementation details. Same data, shaped for the right reader. This took about 30 seconds to generate — compared to the hour or more it typically takes to compile a status report manually. And because it pulls directly from the source of truth in ADO, it's always current.

Part 3: "Tools & Resources"

~10 minutes

Slide 22: Your AI Toolkit

Tool	Purpose	Models
GitHub Copilot (IDE)	In-editor AI assistance, plan mode, implementation	GPT-5.3 Codex, Claude Opus 4.6
GitHub Copilot CLI	Terminal-based AI engineering	GPT-5.3 Codex, Claude Opus 4.6
Azure DevOps MCP Server	Epic/Story management, linking, context	—
GitHub MCP Server	Repository context, PR management, code search	—
GitHub Copilot PR Reviews	Automated pull request review	—

SPEAKER NOTES

Let's talk about the tools available to you. GitHub Copilot — both in your IDE and the CLI version — is your primary AI engineering tool. Make sure you're using the latest models: GPT-5.3 Codex and Claude Opus 4.6. These are dramatically more capable than what was available even 6 months ago. The Azure DevOps MCP server is what connects Copilot to our ADO work items — it's how Copilot reads Epics, Stories, and acceptance criteria. And GitHub Copilot PR reviews provide automated code review on every pull request. These three tools together form the backbone of the workflow we just discussed.

Slide 23: Customizing Copilot

Custom Instructions give Copilot context about how *you* and *your team* work:

Scope	File / Location	Example Use
Repository	<code>.github/copilot-instructions.md</code>	Project conventions, architecture overview, build commands
Path-specific	<code>.github/instructions/*.instructions.md</code> with <code>applyTo</code> globs	Targeted rules for file types or directories
Personal / Global	<code>global-copilot-instructions.md</code> in your platform config dir	Your preferences across all projects

Priority order: Personal > Repository > Organization

Skills are folders of instructions, scripts, and resources that Copilot loads when relevant:

Type	Location	Example
Project skills	<code>.github/skills</code> or <code>.claude/skills</code> in a repo	AMN brand guidelines, Terraform standards
Personal skills	<code>~/.copilot/skills</code> or <code>~/.claude/skills</code>	Your cross-project utilities
Community skills	<code>anthropics/skills</code> , <code>github/awesome-copilot</code>	Pre-built open-source skill packs

SPEAKER NOTES

Before we look at a specific plugin, I want to cover something foundational — how you customize Copilot to work the way YOU want. There are two mechanisms: custom instructions and skills. Custom instructions come in three scopes. Repository-level instructions live in `.github/copilot-instructions.md` — these apply to every Copilot interaction in that repo. Path-specific instructions use glob patterns to target specific file types or directories. And personal instructions in your global config apply your

preferences across every project. Priority goes personal, then repository, then organization. Skills take this further — they're folders of instructions, scripts, and resources that Copilot loads contextually. You can have project skills in your repo, personal skills in your home directory, and community skills from open-source collections. This is actually the foundation of our AMN Plugin Marketplace — Slide 25. The marketplace IS a collection of these skills, packaged and shared across the organization.

Slide 24: Superpowers Plugin

github.com/obra/superpowers

- Makes Copilot **ask more questions** before acting
- Encourages **thorough planning** before implementation
- Results in more **thoughtful, well-reasoned** code
- Easy to install — works with GitHub Copilot CLI

*Thank you to **Rick Alvarez** for the recommendation*

SPEAKER NOTES

I want to call out a specific plugin that I've found incredibly useful: Superpowers, by Jesse Vincent. The link is on the slide. What it does is simple but powerful — it modifies Copilot's behavior to ask more questions before jumping into implementation. It encourages deeper planning and more thorough consideration of edge cases. The result is more thoughtful, well-reasoned code. I want to give credit to Rick Alvarez for recommending this to me — it's made a real difference in my workflow. I'd encourage all of you to try it.

Slide 25: AMN AI Engineering Plugin Marketplace

Our AI Engineering team is building a **shared plugin marketplace** with AMN-specific skills:

Plugin	Description
Brand Guidelines	AMN colors, typography, component styles
Azure Naming Conventions	Resource naming and tagging standards
Terraform IaC Standards	CCOE module usage, repository structure, versioning
AMNLS Git Workflow	Branching strategy, PR conventions for AMNLS
Observability Standards	New Relic integration patterns
Infrastructure Best Practices	Azure architecture and patterns

Goal: Consistent, compliant output from AI across all teams

Install the Marketplace:

```
/plugin marketplace add AMNEngineering/amn-ai-plugin-marketplace
```

SPEAKER NOTES

Our AI Engineering team is building something I'm really excited about — a shared plugin marketplace for AMN-specific AI skills. These are plugins you install into Copilot that give it knowledge about our specific standards. Brand guidelines, Azure naming conventions, Terraform IaC standards, our git branching workflow, observability standards with New Relic — all distilled into skills that any engineer can use. The goal is that when any engineer at AMN uses Copilot to generate code, it automatically follows our conventions. No more "oh, I forgot the naming convention" or "what's the tagging standard again?" The AI just knows. We're rolling these out incrementally and I'd love feedback on what additional skills would be most useful for your teams.

Part 4: "The Hard Truth"

~10 minutes

Slide 26: "AI Made Writing Code Easier. Engineering Got Harder."

Source: Ivan Turkovic — *"AI Made Writing Code Easier. It Made Engineering Harder."*

- The paradox: AI **lowers the barrier** to code production
- But engineers report **increased burnout**, not decreased
- Why? Because the *scope* of what's expected has expanded dramatically

SPEAKER NOTES

Now I want to shift to some hard truths. Ivan Turkovic wrote an article with a title that I think perfectly captures the paradox we're all feeling: "AI Made Writing Code Easier. It Made Engineering Harder." How can that be? If AI writes the code for us, shouldn't our jobs get easier? The answer is nuanced, and I think we owe it to ourselves to be honest about this.

Slide 27: The Numbers

Statistic	Source
83% of AI-using workers report increased workload	Harvard Business Review
67% spend more time debugging AI-generated code	Harness Developer Survey
68% spend more time reviewing AI code than human-written code	Harness Developer Survey
Engineer burnout: 62% vs leadership: 38%	Turkovic, citing industry surveys

SPEAKER NOTES

Let's look at the numbers. Harvard Business Review found that 83% of workers using AI report their workload has *increased*, not decreased. The Harness developer survey found that 67% of developers spend more time debugging AI-generated code than code they write themselves. And 68% say they spend more time reviewing AI-written code than human-written code. Engineer burnout is at 62% — compared to 38% for leadership. These aren't abstract statistics. This is what many of you are probably experiencing right now. If you've felt like AI was supposed to make things easier but somehow your workload increased, you're not alone. You're in the majority.

Slide 28: The Identity Shift

"Your value is not in the code you write anymore — it's in how well you direct the systems that write it for you."

- Engineers became engineers to **write code** — that identity is being disrupted
- New high-value skills:
 - **System design** and architectural thinking
 - **Specification writing** and requirements precision
 - **Review discipline** — reading and evaluating code you didn't write
 - **AI collaboration** — effectively directing AI tools
 - **Domain expertise** — understanding healthcare staffing deeply

SPEAKER NOTES

Here's the deepest challenge, and I want to name it honestly. Most of us became engineers because we love writing code. That's our identity. And that identity is being disrupted. Your value is increasingly not in the code you write — it's in how well you direct the systems that write code for you. That's a hard pill to swallow. But look at what the new high-value skills are: system design, specification writing, review discipline, AI collaboration, and domain expertise. These are all deeply engineering skills. They're arguably **harder** than writing code. You need to understand systems at a deeper level to specify them precisely. You need stronger architectural thinking when you're reviewing someone else's — or something else's — implementation decisions.

Slide 29: Expanding Roles, Unclear Boundaries

Engineers now handle responsibilities previously distributed across teams:

- **Product thinking** — writing specifications requires product judgment
- **Architecture** — directing AI requires architectural vision
- **Code review** — reviewing AI output at scale
- **Testing oversight** — validating AI-generated tests are meaningful
- **Deployment awareness** — understanding the full pipeline

We need to acknowledge this and set realistic expectations

SPEAKER NOTES

The other hard truth is that engineering roles are expanding. When you write specifications, you're doing product work. When you direct AI implementation, you're making architectural decisions. When you review AI output, you're doing QA. When you validate tests, you're doing test engineering. These responsibilities were previously distributed across multiple roles and teams. Now they're concentrating on individual engineers. That's a lot. And I think we as an organization need to acknowledge that and set realistic expectations. The AI makes code production faster, but the total engineering effort — specification, planning, review, validation — may not decrease. It may increase, especially during this transition period. We need to be honest with ourselves and with our leadership about that.

Slide 30: Closing — A Hard But Hopeful Transition

- This will be **difficult** — acknowledge that honestly
- But engineers who adapt will be **extraordinarily productive**
- Previous "obsolescence predictions" have always proved wrong
 - Compilers, IDEs, Stack Overflow, open-source frameworks — none of them replaced engineers
- The role is **evolving**, not disappearing
- AMN is investing in **tools and skills** to support you through this shift

Empowering the Future of Care

That includes empowering our engineers.

SPEAKER NOTES

I want to close honestly. This transition is going to be difficult. It's going to be uncomfortable. Some days the AI will produce something amazing and you'll feel like the future is here. Other days it'll produce garbage and you'll spend two hours debugging something you could have written correctly in 20 minutes. Both of those experiences are normal. But I genuinely believe that the engineers who lean into this will be extraordinarily productive. Every previous "obsolescence prediction" — compilers will replace programmers, IDEs will replace programmers, Stack Overflow will replace programmers, open-source will replace programmers — every single one has been wrong. The role evolves. It doesn't disappear. AMN is investing in the tools, the skills, and the infrastructure to support you through this. We're building the plugin marketplace. We're rolling out MCP integrations. We're developing training and best practices. Our tagline is "Empowering the Future of Care" — and I want you to know that includes empowering our engineers. Thank you. I'd love to take questions.

References

Source	Link
Matt Shumer — "Something Big Is Happening"	shumer.dev/something-big-is-happening
Michael Truell — "The Third Era of AI Software Development"	cursor.com/blog/third-era
Citrini Research — "The 2028 Global Intelligence Crisis"	citriniresearch.com/p/2028gic
David Ondrej — "The Thin Middle Squeeze"	x.com/DavidOndrej1/status/2019126831761572169
VSDD Methodology	gist.github.com/dollspace-gay/d8d3bc3ecf4188df049d7a4726bb2a00
Ivan Turkovic — "AI Made Writing Code Easier. It Made Engineering Harder."	ivanturkovic.com/2026/02/25/ai-made-writing-code-easier-engineering-harder/
Harvard Business Review — AI workload study	hbr.org/2026/02/ai-doesnt-reduce-work-it-intensifies-it
Harness — Developer survey on AI code review	prnewswire.com/.../harness-releases-its-state-of-software-delivery-report...
METR — AI capability tracking	metr.org/time-horizons/
Superpowers Plugin	github.com/obra/superpowers
GitHub Docs — Repository Custom Instructions	docs.github.com/en/copilot/.../add-repository-instructions
GitHub Docs — About Agent Skills	docs.github.com/en/copilot/concepts/agents/about-agent-skills

Appendix: Diagram Notes

All diagrams in this presentation are **custom React SVG components** with:

1. **960x540 viewBox** (16:9) for consistent scaling within slides
2. **AMN brand colors** applied as TypeScript constants
3. **Shared primitives** (FlowNode, FlowArrow, SvgContainer) for consistency
4. **Zero external diagram dependencies** — pure SVG rendered by React

Color Reference (AMN Brand)

Color	Hex	Usage in Diagrams
Navy Blue	#003C69	Primary elements, human work
Purple	#6639B7	AI-driven steps
Teal	#008B95	Process/git steps
Royal Blue	#005BBB	Human review
Cyan	#0098DB	Current/active items
Green	#34B233	Success/completion
Red	#D32F2F	Issues/fixes needed